
Python 金融数据分析

第 7 章 数据输入输出

主讲人： 王伟冠, 吕文俊
院 系： 经济学院金融系

上海大学

2026 年 9 月 24 日



1. 从同一张表认识文件格式
2. 按格式学习操作特点
3. Parquet 与列式存储
4. 同源数据的读取速度实验
5. 综合案例与练习



1. 从同一张表认识文件格式

2. 按格式学习操作特点

3. Parquet 与列式存储

4. 同源数据的读取速度实验

5. 综合案例与练习



- ① 观察同一套虚拟行情：不同文件如何表达同一份数据？
- ② 制作虚拟行情：CSV、Excel、Parquet 三种版本。
- ③ 按格式学习操作：类型、分块、嵌套、工作表、选列。
- ④ 先验证读回结果，再比较文件体积与读取耗时。
- ⑤ 理解 Parquet 的列式存储，并用选列实验检查效果。

本章产物

配套 Notebook 生成可打开的样本文件与实验结果；最后用真实行情完成“读入 → 计算 → 保存”。

格式	组织方式	常见任务与操作重点
CSV	分隔符文本，一张平面表	软件间交换；编码、类型、分块
Excel	工作簿中的多个工作表	人工整理和报告；选表、选列
Parquet	带模式信息的列式二进制	分析中间表；类型、压缩、按列读取

课堂观察

CSV 用记事本看内容；Excel 用表格软件看工作表；Parquet 用程序看模式和数据，不用文本编辑器解释二进制。

- 全章格式示例使用同一套行情，先查看前 6 行理解字段。
- 虚拟行情：固定随机种子 7；每行是一条分钟观测。
- 8 列：日期、代码、市场、价格、成交量、买价、卖价、评分。
- 代码含前导零，市场含中文；评分每隔 17 行缺失一次。
- 同一个 DataFrame 写出三种格式，全部不额外保存索引。

文件直接放在 data 中

data/quotes.csv / .xlsx / .parquet：同一份 20,000 行数据。
用 head(6) 查看小样本，不额外建目录。
数据仅为教学模拟，日期序列并非真实交易日历。

```
1 # Notebook 已设置 CHAPTER, 并导入 sys
2 sys.path.insert(0, str(CHAPTER / "scripts"))
3 from prepare_data import prepare
4 quotes, DATA = prepare(20_000)
5 quotes.head(6)
```

- scripts/prepare_data.py 包含生成规则和三种写法。
- 每次从同一张表写出全部文件；生成、写入不进入读取计时。
- 所需 pandas、numpy、openpyxl、pyarrow 使用课程环境。
- 也可在课前运行脚本；课堂直接查看生成文件。

```
1 # DATA 在数据准备步骤中设置; 不另造一份示例数据
2 with open(DATA / "quotes.csv", encoding="utf-8") as f:
3     for _ in range(7): # 表头 + 前 6 行
4         print(f.readline().rstrip())
```

- 第一行是列名，后面每行是一条行情记录。
- with 自动关闭文件；中文文本明确使用 UTF-8。
- r 读取（默认），w 覆盖，a 追加。
- 接下来仍读取这个文件，用 pandas 还原为 DataFrame。



同一份行情，三种文件预览

基于实际生成文件绘制的教学界面 · 每个文件含相同记录 · 只展示前 3 行

CSV | data/quotes.csv | 文本视图

```
date,symbol,market,price,volume,bid,ask,score
2025-01-01 00:00:00,000001,模拟市场甲,95.04,8049,95.03,95.05,
2025-01-01 00:01:00,000002,模拟市场甲,66.25,5299,66.24,66.26,0.486408
2025-01-01 00:02:00,000003,模拟市场乙,71.57,7942,71.56,71.58,-0.203099
```

Excel | data/quotes.xlsx | 工作表 quotes

date	symbol	market	price	volume	bid	ask	score
2025-01-01 00:00:00	000001	模拟市场甲	95.04	8049	95.03	95.05	缺失
2025-01-01 00:01:00	000002	模拟市场甲	66.25	5299	66.24	66.26	0.486408
2025-01-01 00:02:00	000003	模拟市场乙	71.57	7942	71.56	71.58	-0.203099

Parquet | data/quotes.parquet | 解码后的表格视图

date	symbol	market	price	volume	bid	ask	score
2025-01-01 00:00:00	000001	模拟市场甲	95.04	8049	95.03	95.05	缺失
2025-01-01 00:01:00	000002	模拟市场甲	66.25	5299	66.24	66.26	0.486408
2025-01-01 00:02:00	000003	模拟市场乙	71.57	7942	71.56	71.58	-0.203099

三种预览均展示相同的前 3 行、全部 8 列，列的顺序一致。

Parquet 的表格来自读取器；原始文件是二进制，不能直接当文本浏览。

1. 从同一张表认识文件格式
- 2. 按格式学习操作特点**
3. Parquet 与列式存储
4. 同源数据的读取速度实验
5. 综合案例与练习



```
1 quotes.to_csv(DATA / "quotes.csv", index=False,  
2               encoding="utf-8")  
3 back = pd.read_csv(DATA / "quotes.csv",  
4                   dtype={"symbol": str},  
5                   parse_dates=["date"], encoding="utf-8")
```

- CSV 不保存 pandas 模式：000001 可能被推断为整数 1。
- dtype 指定代码为字符串；parse_dates 解析日期。
- sep 指定分隔符，na_values 约定缺失标记。
- index=False 不写索引；如果日期在索引中，应明确保留它。

```
1 small = pd.read_csv(DATA / "quotes.csv",  
2                      usecols=["price", "volume"])  
3 total = 0  
4 for chunk in pd.read_csv(DATA / "quotes.csv",  
5                           usecols=["volume"], chunksize=5000):  
6     total += chunk["volume"].sum()  
7 assert total == quotes["volume"].sum()
```

- `usecols`: 少构造几列，但仍需扫描 CSV 文本。
- `chunksize`: 每次处理部分行，适合求和、计数等任务。
- 将全部块重新拼接，仍会回到整表占用内存的情况。
- 字段内也可能有逗号或换行，不要用简单字符串切分替代解析器。

```
1 summary = quotes.groupby("symbol", as_index=False)["volume"].sum()
2 with pd.ExcelWriter(DATA / "report.xlsx",
3     engine="openpyxl") as writer:
4     quotes.head(6).to_excel(writer, sheet_name="行情", index=False)
5     summary.to_excel(writer, sheet_name="汇总", index=False)
6 back = pd.read_excel(DATA / "report.xlsx",
7     sheet_name="汇总", dtype={"symbol": str})
```

不是连续对同一路径写两次

两次独立的 `to_excel("report.xlsx")` 默认会重建文件, 前一个工作表会被覆盖。

- `sheet_name` 选工作表, `usecols` 选列。
- 用 `ExcelFile.sheet_names` 查看工作簿中的表名。
- `.xlsx` 的表格读取在本章明确使用 `openpyxl` 引擎。
- `pandas` 读写数据表, 不负责完整保留原工作簿的图表、样式与公式计算行为。
- 大表反复分析前应实测: 工作簿结构的解析本身有开销。

交付习惯

给同事阅读的汇总表可以用 Excel; 计算中间表可以另存 Parquet。

1. 从同一张表认识文件格式
2. 按格式学习操作特点
- 3. Parquet 与列式存储**
4. 同源数据的读取速度实验
5. 综合案例与练习



代码	价格	成交量
000001	12.5	200
000002	20.0	300
000003	18.2	100

按行组织的示意

[000001, 12.5, 200] [000002, 20.0, 300] [000003, 18.2, 100]

按列组织的示意

代码: [000001, 000002, 000003]

价格: [12.5, 20.0, 18.2] 成交量: [200, 300, 100]

只计算平均价格时, 列式布局有机会少读另外两列。



一个 Parquet 文件的内部

实际元数据来自 data/quotes.parquet · 结构示意图，不按字节比例绘制

20,000 行 × 8 列 | 4 个行组 | SNAPPY

文件中的行组和列块不是操作系统文件夹；读取器通过元数据定位它们。

行组 0: 5,000 行

date	symbol	market	price	volume	bid	ask	score
------	--------	--------	-------	--------	-----	-----	-------

行组 1: 5,000 行

date	symbol	market	price	volume	bid	ask	score
------	--------	--------	-------	--------	-----	-----	-------

行组 2: 5,000 行

date	symbol	market	price	volume	bid	ask	score
------	--------	--------	-------	--------	-----	-----	-------

行组 3: 5,000 行

date	symbol	market	price	volume	bid	ask	score
------	--------	--------	-------	--------	-----	-----	-------

文件尾部：元数据 (Footer)

模式 / 类型 · 行组行数 · 列块位置 · 各列统计信息

读取 price、volume：在各行组中定位这两列；每个列块内部再分成页。

显示前 4 个行组；共 4 个。页的编码和压缩细节未按比例展开。



多个 Parquet 文件如何放进目录？

实际目录：data/quotes_by_market/ · 按 market 分区 · 目录名保存分区值

目录视图 | 真实文件名

data/

quotes.parquet ← 单个完整表文件

quotes_by_market/

market=模拟市场乙/

part-0.parquet

6,666 行 · 253.5 KiB

market=模拟市场甲/

part-0.parquet

13,334 行 · 499.9 KiB

选中文件 | 内容检查

market=模拟市场乙

part-0.parquet / 7 列

文件内的列：

date : timestamp[ns]

symbol : string

price : double

volume : int64

bid : double

ask : double

score : double

market 从目录名恢复，不在文件内重复存。

读取逻辑

读数据集根目录 → 合并两个分区；筛选市场甲 → 可以跳过市场乙文件。

普通三格式计时时仍使用 data/ 下的单文件；这个目录仅用于分区教学。

这是 Hive 风格目录分区示例；单个 .parquet 文件不要求这样的目录层级。

```
1 # 单个文件
2 one = pd.read_parquet(DATA / "quotes.parquet")
3 # 两个市场分区，合起来仍是同一张表
4 all_markets = pd.read_parquet(DATA / "quotes_by_market")
5 # 根据目录名，跳过另一个市场的文件
6 market_a = pd.read_parquet(DATA / "quotes_by_market",
7     engine="pyarrow", filters=[("market", "==", "模拟市场甲")])
```

- market= 值是目录分区；行组在文件内部，不是文件夹。
- 本例分区列从目录名恢复；读回后行顺序和分类类型可能变化。
- 分区示例只解释组织方式，不混入三种单文件的速度比较。

```
1 quotes.to_parquet(DATA / "quotes.parquet", index=False,  
2                   engine="pyarrow", compression="snappy",  
3                   row_group_size=5000)  
4 back = pd.read_parquet(DATA / "quotes.parquet",  
5                       engine="pyarrow")  
6 back.dtypes
```

- 本例日期、字符串代码和数值类型随模式保存。
- 20,000 行按每组 5,000 行写入，得到 4 个行组。
- `index=False` 表示不额外保存索引；需要的索引应转成列。
- 压缩降低体积但需要解压；速度依赖数据和具体配置。

```
1 # 读取时只要两列
2 small = pd.read_parquet(DATA / "quotes.parquet",
3                           engine="pyarrow",
4                           columns=["price", "volume"])
5 # 对照：先读取整表，再保留两列
6 wide = pd.read_parquet(DATA / "quotes.parquet", engine="pyarrow")
7 small_after = wide[["price", "volume"]]
```

- 两者返回相同内容，读取和解码的工作量却可能不同。
- 列很多、只取少数列时，通常更值得在读取时指定列。
- 小文件中初始化成本可能更明显，必须用实测检查。

```
1 selected = pd.read_parquet(DATA / "quotes.parquet",  
2     engine="pyarrow", columns=["symbol", "price"],  
3     filters=[("symbol", "==", "000001")])
```

- pyarrow 执行条件筛选；条件允许时，统计信息帮助跳过行组。
- 若每个行组都含目标代码，可能无法跳过任何行组。
- 分区目录可帮助跳过文件；本章同时提供两市场的分区示例。
- Parquet 不提供 Excel 的工作表或数据库的逐行更新接口。
- 本章修改后重写文件；大数据集可通过增加分区文件扩展。



1. 从同一张表认识文件格式
2. 按格式学习操作特点
3. Parquet 与列式存储
- 4. 同源数据的读取速度实验**
5. 综合案例与练习



- ① 同一张原始表、同样行数、同样列顺序，不混用过滤前后数据。
- ② 检查日期类型与精度、证券代码前导零、缺失位置。
- ③ 浮点数采用合理容差，不要求文本往返逐位相等。
- ④ 验证通过后才比较速度；不能把“漏读了内容”当成提速。

Notebook 的核对方法

先统一日期精度和字符串表示，再使用

```
pd.testing.assert_frame_equal(..., check_exact=False,  
rtol=1e-9, atol=1e-8)。
```

标准化与断言在计时外完成；读取参数的解析成本在计时内。

- 问题：同一份 20,000 行 $\times$ 8 列数据，完整读成 DataFrame 要多久？
- 生成、写入与验证不计；计时包围一次读取调用。
- 先预读，再重复 5 轮；每轮随机调整三种格式的顺序。
- 保存每次耗时，报告中位数、最小值、最大值与文件体积。
- 记录 Python、pandas、pyarrow、openpyxl 与操作系统版本。

明确实验边界

未清空操作系统缓存，是有缓存条件下的端到端读取实验；不是磁盘带宽或冷启动测试，也没有测量峰值内存。

```
1 from time import perf_counter
2 start = perf_counter()
3 back = pd.read_parquet(DATA / "quotes.parquet", engine="pyarrow")
4 elapsed_ms = (perf_counter() - start) * 1000
5 print(elapsed_ms)
```

- 不把打印、生成、写文件或一致性断言放进计时区间。
- 单次波动大，Notebook 将三种读取器放入同一重复计时函数。
- 每次得到完整 DataFrame 后再停止计时，随后释放结果。
- 耗时用毫秒，文件体积用 KiB (1024 字节)。

- Notebook 输出：格式、中位数、最小值、最大值、文件体积。
- 哪个最快？哪个最小？耗时波动是否会影响你的判断？
- CSV 本例不压缩；xlsx 容器有压缩；Parquet 用 Snappy。
- 因此比较的是这些常用配置，不是只改变“按行或按列”。
- 不填固定的“实测大小”或通用排名：以本次运行结果为准。

第二组实验：只需要价格与成交量

比较 CSV 的 `usecols`、Parquet 的 `columns`、Parquet 全表后切列。先确认相同结果，再重复计时。

- 文本格式需要将字符解析为数字、日期与缺失值。
- Excel 还要解释工作簿和单元格结构。
- Parquet 需要读元数据、解码与解压；选列可减少工作量。
- 小文件可能由初始化开销主导；宽表的选列收益可能更明显。
- 缓存、硬件、引擎、压缩、数据类型和重复率都会改变结果。

结论写法

写“在这台机器、这份数据和这组参数下……”。不要写“二进制永远更快”或“某格式永远最小”。

1. 从同一张表认识文件格式
2. 按格式学习操作特点
3. Parquet 与列式存储
4. 同源数据的读取速度实验
- 5. 综合案例与练习**



```
1 raw = pd.read_csv(DATA / "eod_data.csv", index_col=0,  
2                   parse_dates=True).dropna(subset=["SPY"])  
3 rets = np.log(raw / raw.shift(1))  
4 vol = (rets["SPY"].rolling(21).std() * np.sqrt(252)).dropna()  
5 table = vol.rename("vol21").rename_axis("date").reset_index()  
6 table.to_parquet(DATA / "spy_vol21.parquet", index=False)  
7 table.to_excel(DATA / "spy_vol21.xlsx", index=False)
```

- 日期索引显式转成列，让输出结构清楚。
- Parquet 保存中间表，Excel 提供给人工查看。
- Notebook 随后读回并验证：保存成功还要核对内容。

- ① 预览三种文件的前 6 行，找到第一条记录和缺失值的表达方式。
- ② 去掉 CSV 的 dtype，观察代码前导零与日期类型。
- ③ 用一个 ExcelWriter 写两个工作表，读回并打印表名。
- ④ 对 2,000 行和 20,000 行分别重做实验，记录环境、体积和耗时。
- ⑤ 选做：增加 20 个数值列，再比较 Parquet 全表与选两列。
- ⑥ 选做：将第 6 章的 beta 表分别保存为 CSV 与 Parquet，读回核对。

回答时至少包含

任务是什么、数据有多大、参数是什么、结果如何、结论适用到哪里。

- CSV：文本交换，明确编码和类型；选列、分块控制工作量。
- Excel：以工作表组织，多个表共用一个 Writer。
- Parquet：列式、类型与压缩；在读取时指定列和筛选条件。
- 先验证数据一致，再用重复计时比较，不预设通用冠军。

下章预告

本章比较“读数据”；下一章比较“算数据”：循环、向量化与性能测量。

- 配套 Notebook: `notebooks/07_Input_Output.ipynb`
- 数据生成脚本: `scripts/prepare_data.py`
- 教材基础: 第 9 章^[1]; 新增 API 参照官方文档。
- pandas: `read_csv`
- pandas: `ExcelWriter`
- pandas: `read_parquet`
- Apache Parquet: Overview 与 File Format

官方文档查阅日期: 2026-09-24。实验排名由本地运行产生。



- [1] 伊夫·希尔皮斯科. Python 金融大数据分析 [M]. 2nd ed. 北京: 人民邮电出版社, 2018.



Thanks for your attention!

Q & A

